

SDK / стек

Название приложения	Chicken road game
Android Gradle Plugin	8.5.2
minSdk	26
targetSdk	35
Kotlin	да 2.0.21
Web View	да
Custom Tabs	нет
Рекламные сети	нет
Аналитика	AppsFlyer, Firebase Realtime Database, Install Referrer
Permissions	android.permission.INTERNET, android.permission.ACCESS_NETWORK_STATE, com.google.android.gms.permission.AD_ID, android.permission.WRITE_EXTERNAL_STORAGE (maxSdk=28), android.permission.CAMERA, android.permission.POST_NOTIFICATIONS, android.permission.ACCESS_AD_SERVICES_ATTRIBUTION, com.huawei.appmarket.service.commondata.permission.GET_COMMON_DATA, com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE, com.android.vending.CHECK_LICENSE, com.chicken.road.cerman.fixs.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
Libraries	androidx.compose (UI/Material3), androidx.activity, androidx.navigation, androidx.lifecycle, androidx.datastore, androidx.room, androidx.core, androidx.startup, androidx.fragment, androidx.profileinstaller, com.appsflyer (af-android-sdk 6.14.0), com.google.firebase (common, database), com.google.android.gms (base, tasks, ads-identifier), com.android.installreferrer, okhttp3, okio, kotlinx.coroutines, kotlinx.serialization, com.pairip (licensecheck), kotlin-stdlib
Подозрительные домены	config.ru
SharedPreferences	DataStore app_state: URL страницы (page), метка времени, флаг согласия Privacy Policy (agreed)
Есть ли клоака	да
Подозрительные слова	uniwebview, checkUrl, serverCheckUrl, myGame, consent, referrer, gaid, bridge

Проверка домена: config.ru

Домен	config.ru
VirusTotal URL	https://www.virustotal.com/gui/domain/config.ru
Детекции	0/91 (malicious=0, suspicious=0)
Security vendors' analysis	ниже построчно, как на VirusTotal
Abusix	Clean
Acronis	Clean
ADMINUSLabs	Clean
AILabs (MONITORAPP)	Clean
AlienVault	Clean
alphaMountain.ai	Clean
Antiy-AVL	Clean
BitDefender	Clean
Blueliv	Clean
Certego	Clean
Chong Lua Dao	Clean
CINS Army	Clean
CMC Threat Intelligence	Clean
CRDF	Clean
CTX AI	Clean
Cyble	Clean

CyRadar	Clean
desenmascara.me	Clean
Dr.Web	Clean
EmergingThreats	Clean
Emsisoft	Clean
ESET	Clean
ESTsecurity	Clean
Forcepoint ThreatSeeker	Clean
Fortinet	Clean
G-Data	Clean
Google Safe Browsing	Clean
GreenSnow	Clean
Heimdal Security	Clean
IPsum	Clean
Juniper Networks	Clean
Kaspersky	Clean
LevelBlue	Clean
Lionic	Clean
Malwared	Clean
MalwarePatrol	Clean
OpenPhish	Clean
Phishing Database	Clean
Phishtank	Clean
PREBYTES	Clean
Quick Heal	Clean
Quttera	Clean
Scantitan	Clean
SCUMWARE.org	Clean
Seclookup	Clean
Sophos	Clean
StopForumSpam	Clean
Sucuri SiteCheck	Clean
ThreatHive	Clean
URLhaus	Clean
Viettel Threat Intelligence	Clean
ViriBack	Clean
VX Vault	Clean
Webroot	Clean
Xcitium Verdict Cloud	Clean
Yandex Safebrowsing	Clean
ZeroCERT	Clean
0xSI_f33d	Unrated
AlphaSOC	Unrated
ArcSight Threat Intelligence	Unrated
AutoShun	Unrated
Axur	Unrated
Bfore.Ai PreCrime	Unrated

Bkav	Unrated
ChainPatrol	Unrated
Cluster25	Unrated
Criminal IP	Unrated
CSIS Security Group	Unrated
Cyan	Unrated
DNS8	Unrated
Ermes	Unrated
Fortra	Unrated
GCP Abuse Intelligence	Unrated
GreyNoise	Unrated
Gridinsoft	Unrated
Guardpot	Unrated
Hunt.io Intelligence	Unrated
Lumu	Unrated
MalwareURL	Unrated
Mimecast	Unrated
Netcraft	Unrated
PhishFort	Unrated
PrecisionSec	Unrated
SafeToOpen	Unrated
Sansec eComscan	Unrated
SecureBrain	Unrated
Snort IP sample list	Unrated
SOCRadar	Unrated
URLQuery	Unrated
VIPRE	Unrated
ZeroFox	Unrated
Куда редиректит	нет
Что выводит (кратко)	не удалось открыть (ReadTimeout)
Где припаркован	регистратор: RU-CENTER-RU

Как работает клоака

В начале

Снаружи приложение выглядит как безобидный трекер курятника / «Chicken Road Game»: на экране — Compose-интерфейс про кур, корм, яйца, прививки и статистику. Внутри при каждом запуске тихо работает отдельный модуль roost: он спрашивает у Firebase и у своего проверочного сервера, показывать ли эту «белую» игру или открыть внешнюю ссылку в полноэкранном WebView (встроенный браузер внутри приложения).

Обман в том, что модератору магазина и «неподходящему» трафику показывают обычную ферму-утилиту, а «нужному» пользователю сервер может отдать другой URL — тогда человек сразу попадает на внешнюю страницу (оффер / лендинг / политику с кнопкой Ассерт). Финальный адрес не зашит в APK: его каждый раз присылает удалённый check-сервер (сейчас в конфиге Firebase это `https://cerman.space`), в ответе JSON вида `{url, time}`.

Кому что показывают

Белая страница / обычный режим: пользователь видит локальное приложение-ферму (экраны Hub, Flock, Eggs, Feed, Health, Stats). Сюда попадают, если проверочный сервер вернул пустой url, если удалённый конфиг выключен, если произошла ошибка запроса, либо после принятия Privacy Policy

(флаг `agreed`) — тогда приложение остаётся в безопасном виде.

Чёрный / боевой режим: открывается `RoostPageActivity` — полноэкранный `WebView` с JavaScript, cookies, загрузками и JS-мостом. Туда попадают, когда `check-API` вернул непустой url. Если в ссылке есть специальный маркер «`consent`» (строка из `native`-библиотеки `libroostio`), сверху показывают заголовок Privacy Policy и кнопку Акцепт; иначе `WebView` ведёт себя как «липкий» браузер оффера и URL сохраняется для следующих запусков.

Как приложение решает, кого куда пустить

- 1) Firebase Realtime Database (удалённый конфиг). Смотрит узел `appConfig`: флаги `myGame` (`active`), `serverCheckUrl` (адрес проверки), `title` (ключ `AppsFlyer`). Условие: `myGame=true` и `serverCheckUrl` не пустой → идём на серверную проверку; иначе остаёмся на белой ферме или на ранее сохранённом URL.
 - 2) Серверный check по `serverCheckUrl` (сейчас `serman.space`). Клиент шлёт GAID, device UUID, `install referrer` и `AppsFlyer UID`. На сервере (в коде приложения списков стран/ботов нет) принимают решение и отвечают `JSON {url, time}`. Непустой url → `WebView`; пустой/ошибка → белая ферма.
 - 3) User-Agent. Перед запросом и в `WebView` функция `tidyAgent` убирает признаки «; wv»)» и «Version/4.0 », чтобы трафик меньше выглядел как Android `WebView` для фильтров на стороне лендинга/сервера.
 - 4) Флаг `agreed` в `DataStore`. После Акцепт на странице Privacy Policy пишется `agreed=true`, `MainActivity` перезапускается, и дальше показывается белый режим (постоянный «чистый» путь для `review/organic`).
 - 5) Кэш `pageUrl`. Успешный боевой URL сохраняется вместе с `time`; при следующих запусках, если удалённый конфиг недоступен, приложение может снова открыть сохранённую страницу.
- Гео/VPN/datacenter-фильтры в Java-коде клиента не видны — решение отдано серверу `serman.space`. Явных строк `cloak/gambling/casino` в APK нет: чувствительные имена ключей спрятаны в `native`-библиотеке `libroostio` (`RoostBank.unpack`).

Цепочка по шагам

- Шаг 1. Стартует `process` → `Pairip Application`-обёртка и `BroilerApp` инициализируют `Firebase` → зачем: подготовить удалённый конфиг → система готова читать `appConfig`.
- Шаг 2. `MainActivity` показывает `splash` и вызывает `RoostGate.decide()` → зачем: скрыть проверку за заставкой → пользователь пока не видит ни ферму, ни оффер.
- Шаг 3. `RoostFeed` грузит `Firebase appConfig` (`myGame`, `serverCheckUrl`, `title`) → зачем: узнать, включена ли клоака и куда стучаться → получает, например, `serverCheckUrl=https://serman.space`.
- Шаг 4. Собираются GAID, стабильный UUID устройства, `install referrer`; при наличии `title` поднимается `AppsFlyer` → зачем: отдать серверу «отпечаток» установки → идентификаторы уходят в `query`-параметрах.
- Шаг 5. `RoostClient` делает GET на `check-URL` с подчищенным User-Agent → зачем: сервер решит `white/black` → ответ `{url, time}`.
- Шаг 6. Если url пустой → маршрут `Home` → пользователь получает `Compose`-ферму (белая витрина).
- Шаг 7. Если url непустой и похож на `consent/Privacy` → `RoostPageActivity` в режиме согласия → человек видит политику и Акцепт; после Акцепт ставится `agreed` и снова белая ферма.
- Шаг 8. Если url боевой (без `consent`-маркера) → тот же `WebView` без «белого» тулбара, URL пишется в `DataStore` → пользователь остаётся на внешнем лендинге/оффере.
- Шаг 9. Повторные запуски читают `agreed/pageUrl` и снова либо ферму, либо сохранённый `WebView` → зачем: не дергать цепочку зря и закрепить выбранный режим.

Технические уловки

Native-обфускация `libroostio` / `RoostBank`: пути `Firebase`, имена `query`-параметров, маркер `consent` и ключи `DataStore` достаются через `unpack(idx)`, а не лежат открытым текстом. Зачем: усложнить статический поиск клоаки по строкам.

`WebView` + JS-мост (схема `uniwebview`, интерфейс `AndroidDownloader`): лендинг может управлять загрузками и закрытием. Зачем: полноценный «браузер оффера» внутри APK без Custom Tabs.

Подчистка User-Agent (`tidyAgent`): маскирует `WebView`. Зачем: пройти фильтры, которые режут трафик с меткой `wv`.

Package mismatch: в Play указан `com.chicken.road.whale`, в бинарнике — `com.chicken.road.serman.fixs`. Зачем (по смыслу поставки): смена/маскировка идентификатора при дистрибуции; на логику клоаки влияет косвенно.

`network_security_config` разрешает `cleartext` — можно открывать и `http`-лендинги.

Роль подозрительных доменов

config.ru — в domain_checks попал как «домен», но в APK это имя language-split пакета (config.ru.apk / splits0.xml для локали ru), а не хост, к которому ходит клоака. VirusTotal: 0/91, редиректа нет, страница не открылась (ReadTimeout), парковка/регистратор RU-CENTER-RU. К цепочке white/black решения он не подключён.

Реальный «мозг» проверки сейчас — хост из Firebase serverCheckUrl: cerman.space. На шаге 5 клиент запрашивает его и получает динамический url (в момент разбора это была страница telegra.ph Privacy Policy — белый consent-путь). Сам cerman.space в domain_checks не проверялся; он не захардкожен в Java, а приезжает удалённо.

Финал для пользователя

Итоговый сайт всегда динамический: его отдаёт check-API в поле url. В момент анализа сервер отдавал Telegraph Privacy Policy — то есть «мягкий» белый сценарий с Ассерт и дальнейшей фермой. При смене ответа на стороне cerman.space тот же WebView без обновления APK может открыть любой лендинг (казино/беттинг/оффер и т.п.). Белый контент приложения при этом остаётся локальной фермой на Compose и не связан с рекламными сетями (их в клиенте нет).